Guía 1 de 3 — El proyecto de ejemplo, paso a paso

DPL1046 · Clase 8 · Para repasar en casa, a tu ritmo.

En la Clase 8 viste un proyecto integrador **completo**, de principio a fin. Esta guía te deja recorrerlo de nuevo con calma: qué problema resuelve, cómo está organizado, cómo correrlo y qué hace cada archivo. **No es para copiarlo**: es para entender cómo se ve un proyecto que cumple la rúbrica y comparar el tuyo.

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2) con el `.venv` activado y trae lo último con `git pull`.

---

1. El problema que resuelve

Un equipo de compras de un e-commerce necesita saber **cuántas ventas esperar en las próximas 8 semanas**. Sin ese pronóstico se compra de más (dinero congelado en inventario) o de menos (ventas perdidas).

El proyecto usa **dos fuentes de datos**:

- **Olist** (CSV): pedidos (99.441) e ítems de cada pedido (112.650).
- **API del dólar** (`mindicador.cl`): para pasar los precios de reales brasileños (BRL) a pesos chilenos (CLP).

> **Analogía:** es como una cocina. Primero traes los ingredientes (**extraer**), luego los lavas, cortas y cocinas (**transformar**) y al final sirves el plato (**cargar**). A esa secuencia se le llama **ETL**: *Extract*, *Transform*, *Load*.

2. Cómo está organizado

```
proyecto-ejemplo/
├── pipeline.py          ← corre todo, en orden
├── config.yaml          ← rutas y parámetros
├── requirements.txt     ← librerías necesarias
├── README.md            ← cómo usarlo
├── src/
│   ├── extraer.py       ← lee los datos
│   ├── transformar.py   ← calcula
│   └── cargar.py        ← guarda
└── tests/
    └── test_pipeline.py ← 4 pruebas con pytest
```

Una carpeta, un rol. `pipeline.py` no calcula nada: solo llama a las funciones de `src/` en orden, como un jefe de cocina que dice "ahora tú, ahora tú".

3. Cómo correrlo

Con el entorno activo, desde la raíz del repo:

```
cd clases/clase-08/material/proyecto-ejemplo
pip install -r requirements.txt
python pipeline.py
pytest tests -v
```

Debes ver los pasos `1/5` a `5/5`, un **RESUMEN EJECUTIVO** y al final `PIPELINE COMPLETADO`. Después, `pytest` debe mostrar `4 passed`. Los resultados quedan en la carpeta `salida/`.

Si algo falla:

Problema	Qué hacer
`No such file` al correr `python pipeline.py`	Estás en otra carpeta. Haz `cd` a `proyecto-ejemplo` primero (el archivo `config.yaml` usa rutas relativas).
`No encontré 'olist_orders_dataset.csv'`	Faltan los CSV de Olist en `data/raw/` (los de la Clase 1).
Aviso `No se pudo obtener tipo de cambio real`	**No es un error:** no hay internet o la API no respondió. El pipeline usa un valor de respaldo (1 BRL ≈ 180 CLP) y sigue.
`ModuleNotFoundError`	Falta instalar las librerías: `pip install -r requirements.txt` con el `(.venv)` activo.

4. Recorrido por los archivos

`pipeline.py`: el director de orquesta

Lee `config.yaml` y llama a cada paso en orden: extraer → transformar (limpiar, estacionalidad, pronóstico) → cargar → exportar CSV. Cada paso avisa qué hizo con `logger.info(...)` y `logger.success(...)`: es el **diario** del programa (Clase 7).

`src/extraer.py`: una API con plan B

```
        try:
            r = requests.get(url, timeout=10)
            r.raise_for_status()
            valor_usd_clp = r.json()["serie"][0]["valor"]
            ...
        except Exception as e:
            logger.warning(f"No se pudo obtener tipo de cambio real: {e}. Usando
fallback.")
            return 180.0
```

- `requests.get(url, timeout=10)` pide el dólar (como en la Clase 4); si tarda más de 10 segundos, se rinde.
- `raise_for_status()` avisa si la respuesta fue un error.
- `["serie"][0]["valor"]` entra al JSON: lista → diccionario → valor más reciente.
- `except` atrapa **cualquier** falla, la **registra** y devuelve un **valor de respaldo**: el pipeline no se cae.

> **Analogía:** es el plan B del taxi. Si la app no funciona, llamo por teléfono.

`src/transformar.py`: limpiar, combinar y resumir

1. **Filtrar** solo los pedidos entregados: `pedidos[pedidos["order_status"] == "delivered"]` (Clase 5).
2. **Combinar** pedidos con ítems: `pd.merge(..., on="order_id", how="inner")` (Clase 6).
3. **Columna nueva** con el precio en pesos: `ventas["precio_clp"] = (ventas["price"] * tipo_cambio).round(0)`.
4. **Resumir por semana** con `groupby("semana").agg(...)`: pedidos, ítems y ventas (Clase 6).
5. Calcula la **estacionalidad** (índice por mes; 1,0 es un mes promedio) y un **pronóstico simple**: la media de las últimas 8 semanas más una tendencia.

> El pronóstico es simple **a propósito**: un modelo sofisticado no es el objetivo del diplomado, sino un pipeline ordenado y confiable.

`src/cargar.py`: cargar sin duplicar

```
        ya = _semanas_existentes(engine, "ventas_semana")
        nuevas = ventas_semana[~ventas_semana["semana"].isin(ya)]
        nuevas.to_sql("ventas_semana", engine, if_exists="append", index=False)
        ...
        pronostico.to_sql("pronostico", engine, if_exists="replace", index=False)
```

- Primero mira **qué semanas ya están** en la base.
- Se queda solo con las **nuevas** (`~` significa "no") y las agrega con `append`.

- El pronóstico se **reemplaza** siempre, porque cambia en cada ejecución.

Resultado: si corres el pipeline **dos veces**, la segunda dice **0 filas nuevas**. Eso se llama **idempotencia** y está en la rúbrica.

`config.yaml`: parámetros fuera del código

```yaml
datos:
  carpeta: "../../../../data/raw"
pronostico:
  semanas: 8
base_datos:
  uri: "sqlite:///salida/demanda.db"
salida:
  carpeta: "salida"
```

> **Analogía:** el termostato. Cambio la temperatura en el dial, no abro la caldera. Si quiero **12 semanas** de pronóstico, cambio un número aquí y no toco el código. Recuerda: `config.yaml` guarda parámetros (se puede subir a Git); `.env` guarda secretos (**nunca** se sube).

`tests/test_pipeline.py`: pruebas automáticas

Son 4 pruebas con `pytest`: la estacionalidad tiene 12 meses, el pronóstico genera la cantidad de semanas pedida, las fechas del pronóstico son futuras y la carga es idempotente. Cada una usa `assert` para **afirmar** algo que debe ser verdad.

5. Qué punto de la rúbrica cubre cada pieza

Criterio (peso)	Dónde se ve en el ejemplo
Funcionalidad (35%)	Corre completo con `python pipeline.py`, integra CSV y API, resultados reproducibles.
Calidad del código (30%)	Funciones con docstring, `try/except`, logging, 4 tests.
Buenas prácticas (15%)	`config.yaml`, `requirements.txt`, README, carga sin duplicar, sin claves en el código.
Presentación (20%)	Problema claro, demo en vivo, valor de negocio. Mira la Guía 2.

6. Prueba tú mismo (10 minutos)

1. **Cambia un parámetro.** En `config.yaml` cambia `semanas: 8` por `semanas: 12`, corre `python pipeline.py` y abre `salida/pronostico.csv`: ahora tiene 12 filas. No tocaste el código.
2. **Corre dos veces.** En la segunda ejecución, el paso 4 dice `0 filas nuevas`.

3. **Sin internet.** Desconecta el wifi y corre de nuevo: verás el aviso de la API y el pipeline igual termina.
4. **Rompe un test a propósito.** En `tests/test_pipeline.py` cambia el número de semanas esperado, corre `pytest tests -v` y lee con calma qué esperaba y qué obtuvo. Luego devuélvelo.

7. Preguntas de repaso

1. ¿Qué hace `pipeline.py` y por qué no calcula nada?
2. ¿Qué pasa si la API del dólar falla? ¿Dónde queda registrado?
3. ¿Por qué se usa `append` para las ventas y `replace` para el pronóstico?
4. Si quiero pronosticar 12 semanas en vez de 8, ¿qué archivo cambio?
5. ¿Qué afirma `assert len(resultado) == n`?

Respuestas

1. Llama a las funciones de `src/` en orden; el cálculo vive en cada módulo, así cada archivo tiene una sola responsabilidad.
2. Usa un valor de respaldo y lo registra con `logger.warning(...)`.
3. Las ventas históricas no cambian (solo se agregan semanas nuevas); el pronóstico cambia siempre, así que se reescribe.
4. `config.yaml`.
5. Que el pronóstico tiene exactamente `n` semanas.